

NEPALORA — ENGINEERING DOCUMENTATION

Performance, Caching & Cloudinary Optimization

A comprehensive implementation guide for making Nepalora fast, stable, cache-efficient and economical while preserving the existing visual design and user experience.

Platform: Next.js / Vercel / Database / Cloudinary

Scope: Public website performance, media delivery, caching, database efficiency and frontend optimization

Design policy: No visual redesign

1. Objective

Nepalora is designed to become a content-rich Nepal-focused publication containing articles, destinations, travel information, food, culture, events, galleries and other editorial material.

As the content library grows, the website must remain fast even when there are hundreds or thousands of images and many published articles. Performance should therefore be built into the architecture rather than treated as a later cosmetic improvement.

Primary objective: Make every request appropriate for its purpose. Small thumbnails should receive small optimized images. Full-resolution images should only be requested when necessary. Stable public content should be cached. Database queries should be minimal. Non-essential JavaScript should not block the initial page.

The existing design, branding, colors, typography, layouts, navigation, cards, gallery appearance and overall visual identity must remain unchanged.

2. Non-Negotiable Performance Rules

- **No redesign.** Performance improvements must happen behind the existing interface.
- **No unnecessary original-image delivery.** A small UI element must never receive a huge source image unnecessarily.
- **No duplicate requests.** The same data or image should not be fetched repeatedly without a reason.
- **No unnecessary database calls.** Public content should not repeatedly query the database when it can safely be cached.
- **No blocking non-critical content.** Secondary content should not prevent the main page from rendering.
- **No unnecessary JavaScript.** Public pages should not load admin functionality or heavy libraries that they do not need.
- **Measure everything.** Performance changes should be validated using real production requests.
- **Optimize for mobile.** Nepalora must remain usable on slower mobile networks and lower-powered devices.

3. Recommended Request Architecture

The desired architecture is a sequence where each layer performs only the work it actually needs to perform.

```
Visitor
  ↓
Next.js public page
  ↓
Cached / revalidated content
  ↓
Database query only when required
  ↓
Cloudinary CDN
  ↓
Optimized image derivative
  ↓
Browser cache
  ↓
Rendered image
```

The important principle is that a returning visitor should not cause Nepalora to repeat expensive work that has already been completed.

4. Cloudinary Storage Optimization

4.1 Store one useful master asset

Avoid maintaining separate permanent copies of the same photograph for every possible screen size.

- One good-quality source/master image is normally sufficient.
- Generate delivery variants using Cloudinary transformations.
- Do not permanently upload separate thumbnail, mobile, tablet and desktop versions unless there is a real editorial reason.
- Delete abandoned test uploads.
- Delete duplicate images.
- Periodically audit unused assets.

4.2 Use organized folders

Cloudinary assets should use predictable folders.

```
nepalora/
├─ articles/
├─ gallery/
├─ destinations/
├─ food/
├─ events/
├─ places/
├─ site/
└─ misc/
```

Organization makes it much easier to identify unused or duplicated media later.

4.3 Automatic format and quality

Cloudinary should generally deliver the most appropriate format and compression level rather than forcing every browser to download the original JPEG or PNG.

```
f_auto,q_auto
```

Use automatic format and quality delivery where appropriate. This can substantially reduce transfer size while maintaining visual quality.

4.4 Resize according to display purpose

Usage	Recommended approach
Small card	Serve a small derivative appropriate for the card width.
Gallery thumbnail	Serve a thumbnail close to the rendered display size.
Article image	Serve an image appropriate for the article content width.
Hero image	Use a larger derivative only where the design actually displays it large.
Full-screen image	Request the larger image only when the visitor opens it.

Important: CSS resizing does not reduce the downloaded file size. If a 4 MB image is downloaded and CSS displays it at 400px, the browser still downloaded the 4 MB file. The image URL itself must point to an optimized derivative.

5. Cloudinary Bandwidth Optimization

Storage is only one part of Cloudinary usage. A website can have modest storage but consume significant bandwidth if visitors repeatedly download large images.

Reduce bandwidth by:

- Using **f_auto**.
- Using **q_auto**.
- Serving appropriate image widths.
- Using thumbnails for previews.
- Loading large images only when necessary.
- Preventing duplicate requests.
- Allowing CDN caching.
- Removing unnecessary image downloads from hidden components.

6. Gallery Optimization

The gallery currently contains only a small number of images, so the goal is not to create unnecessary complexity. The main objective is to ensure that the page does not wait several seconds before becoming usable.

Recommended gallery flow

```
graph TD;
    A[Database] --> B[Fetch gallery metadata];
    B --> C[Generate optimized thumbnail URLs];
    C --> D[Render gallery immediately];
    D --> E[Browser loads thumbnails];
    E --> F[User selects an image];
    F --> G[Request larger image only when required];
```

Gallery rules

- Do not use original full-size images as gallery thumbnails.
- Do not download full-resolution versions before interaction.
- Do not make the gallery wait for unrelated page data.
- Do not fetch the same gallery data multiple times.
- Give images stable dimensions or aspect ratios.
- Allow images to load independently.

7. Lazy Loading

Lazy loading means that images that are not currently needed are allowed to wait until they approach the user's viewport.

For example, if a page contains 30 article images but the visitor can initially see only three, there is little reason to immediately download all 30.

Recommended behavior

- Above-the-fold primary image: load immediately when appropriate.
- Below-the-fold images: lazy-load.
- Gallery thumbnails: lazy-load where useful.
- Large lightbox image: load after the visitor requests it.

Lazy loading is not a substitute for image optimization. A badly sized image that loads lazily is still a badly sized image. Both techniques should be used together.

8. Next.js Image Optimization

Next.js image handling should be used consistently throughout the public website where compatible with the current architecture.

- Use responsive image sizing.
- Use accurate **sizes** information.
- Use lazy loading for non-critical images.
- Use eager/priority loading only for genuinely important above-the-fold imagery.
- Do not mark every image as priority.
- Prevent layout shifts by reserving the image's dimensions.

Conceptual image strategy

```
Hero image
→ optimized large derivative
→ eager when genuinely critical

Gallery thumbnails
→ optimized small derivatives
→ lazy

Article images
→ appropriate content width
→ lazy below the first viewport

Large image viewer
→ larger derivative
→ loaded on interaction
```

9. Database Optimization

Because image references are stored in the database, database behavior must be checked separately from Cloudinary.

9.1 Select only required fields

Do not retrieve an entire database row when the gallery only needs a few fields.

```
SELECT
  id,
  image_url,
  alt_text,
  sort_order
FROM gallery_images
WHERE published = true
ORDER BY sort_order;
```

Adapt the query to the actual schema.

9.2 Avoid N+1 requests

Do not fetch the gallery and then perform another database request for every individual image.

Whenever possible, retrieve the required metadata efficiently in one query or a small number of predictable queries.

9.3 Index commonly filtered fields

Index columns used frequently in filtering, sorting and relationships.

- Published/status fields.
- Category fields.
- Slug fields.
- Ordering fields where appropriate.
- Foreign keys used in common queries.

Do not blindly index every column. Excessive indexes increase storage and can slow writes.

10. Caching Strategy

Caching is one of the most important optimizations for a content-driven website.

Articles, gallery metadata, categories, navigation and site settings usually do not change every second. There is therefore no reason to perform expensive work for every visitor.

Recommended caching layers

Layer	Cache	Purpose
Browser	Images and static assets	Fast repeat visits.
Cloudinary CDN	Image derivatives	Fast global media delivery.
Next.js / Vercel	Public HTML/data	Reduce repeated server work.
Database/API	Stable public content	Reduce repeated database queries.

11. Public Page Caching

Public articles and gallery pages should be designed to work well with caching and revalidation.

Preferred model

```
Article published
  ↓
Generate / update page
  ↓
Cache public result
  ↓
Visitors receive cached result
  ↓
Article edited
  ↓
Revalidate affected page
  ↓
New version becomes available
```

This approach is preferable to forcing every visitor to perform a fresh database query.

Time-based revalidation

For content that does not need to update instantly, time-based revalidation can be used.

For content where immediate publishing matters, on-demand revalidation after publishing or editing is preferable.

12. Avoid Accidental Dynamic Rendering

One of the most important things to investigate in Next.js is whether an otherwise cacheable public page is accidentally being forced into dynamic rendering.

Review the use of:

- Cookies.
- Request headers.
- Authentication checks.
- Uncached database requests.
- Dynamic server functions.
- Client-side data fetching that could be handled on the server.

Do not make the entire public page dynamic simply because a small component requires dynamic information. Separate dynamic/private functionality from stable public content wherever practical.

13. API Optimization

If the frontend calls API routes to obtain gallery or article information, inspect those routes carefully.

- Do not request the same endpoint multiple times.
- Do not make unnecessary sequential API calls.
- Return only the fields required by the frontend.
- Cache stable public GET responses where appropriate.
- Do not publicly cache private/admin responses.
- Remove unnecessary polling.

14. Client-Side JavaScript

JavaScript can make a website slower even when images and database requests are optimized.

Rules

- Keep public pages as lightweight as possible.
- Do not make the entire application a client component without a reason.
- Use server components where suitable.
- Load heavy components only when needed.
- Use dynamic imports for genuinely heavy features.
- Keep admin-only dependencies out of public bundles.

Potentially heavy dependencies

- Rich text editors.
- Charts.
- Maps.
- Large animation libraries.
- Image editors.
- Admin dashboards.
- Large icon libraries.

15. Prevent Duplicate Requests

Duplicate requests can make a site appear mysteriously slow.

Check for situations where:

- A parent component fetches data and a child fetches it again.

- Multiple **useEffect()** calls request the same data.
- A component is mounted more than once.
- Image URLs are requested by both hidden and visible components.
- Client and server code both request identical content.
- Navigation triggers an unnecessary full reload.

16. Diagnosing the Current 5–7 Second Gallery Delay

A gallery containing only six images should not automatically require five to seven seconds to become usable. Do not guess which service is responsible. Identify the slow request.

Step 1 — Test production

- Open the deployed Vercel website.
- Use an incognito/private browser window.
- Open the gallery.
- Repeat the test several times.
- Compare first-load and repeat-load behavior.

Step 2 — Open Network

In Chrome:

```
Developer Tools
→ Network
→ Reload page
→ Sort by Duration
```

Find the request responsible for the delay.

Step 3 — Determine what is slow

Slow request	Likely area to investigate
Document / page request	Next.js rendering, server processing, database or caching.
API request	Database query, server function or API architecture.
Image request	Image size, transformation, network or delivery path.
JavaScript bundle	Bundle size, client components or dependencies.

17. The Hard-Test Method

Use controlled tests to identify the bottleneck.

Test A — Remove database dependency temporarily

Temporarily render known gallery image data directly.

If the gallery becomes immediately fast, investigate the database/API path.

Test B — Use tiny images temporarily

Replace the gallery images temporarily with very small optimized images.

If the delay disappears, investigate image size and image delivery.

Test C — Disable non-essential components

Temporarily remove analytics, widgets, animations and third-party scripts from the test build.

If performance improves substantially, reintroduce them individually and identify the expensive component.

18. Image Dimensions and Layout Stability

Every image should have a predictable display ratio or dimensions before the image finishes downloading.

This prevents the page from jumping when images arrive.

- Use width/height where appropriate.
- Use an aspect-ratio container when the design requires it.
- Maintain the existing visual proportions.
- Do not allow images to suddenly push content downward.

19. Article Image Strategy

Articles will likely become one of the largest sources of Cloudinary bandwidth as Nepalora grows.

Recommended rules

- Do not upload enormous camera originals unless they are genuinely needed.
- Compress images before upload when practical.
- Use Cloudinary transformations for delivery.
- Use automatic format and quality.
- Use article-content-width derivatives.
- Lazy-load images below the first viewport.
- Do not embed large images directly inside article JSON.

20. Upload Pipeline

The admin media workflow should prevent unnecessary assets from entering the system.

```
Admin selects image
↓
Validate file type
↓
Validate dimensions
↓
Compress / normalize if appropriate
↓
Upload to Cloudinary
↓
Store asset reference + metadata
↓
Render optimized Cloudinary URL
↓
Cache delivery
```

Recommended validation

- Allowed file types.
- Maximum source dimensions.
- Maximum upload size.
- Duplicate detection where practical.
- Clear asset naming.

21. Avoiding Cloudinary Waste

Wasteful approach	Better approach
Upload separate copies for every screen size.	Keep a useful master and generate delivery derivatives.
Serve 5000px images to 400px cards.	Serve a card-sized derivative.
Use PNG for normal photographs.	Use an efficient photographic format.
Download originals then resize using CSS.	Request the appropriate transformed URL.
Keep test and duplicate uploads forever.	Audit and delete unused assets.
Generate many unnecessary transformation variants.	Standardize a small number of delivery purposes.

22. Standard Image Delivery Profiles

Instead of randomly generating different image sizes throughout the application, establish a small number of standard delivery profiles.

Profile	Purpose
Thumb	Small cards, lists and gallery previews.
Card	Medium editorial cards.
Article	Main article content.
Hero	Large primary visual.
Full	Large viewer or user-requested original-quality display.

The exact pixel dimensions should be based on the actual Nepalora components rather than arbitrary numbers.

23. Fonts, Icons and Static Assets

- Load only the font weights actually used.
- Avoid loading unnecessary font families.
- Use optimized SVGs for suitable icons and logos.
- Avoid shipping a huge icon library when only a few icons are required.
- Compress static images.
- Keep the favicon and branding assets lightweight.

24. Third-Party Scripts

Analytics, advertising, social embeds, maps, chat widgets and other external services can become significant performance costs.

- Audit every third-party script.
- Remove scripts that are no longer necessary.
- Do not use multiple tools that perform the same job.
- Delay non-critical scripts when practical.
- Do not allow third-party scripts to block the main content.
- Load heavy embeds only when the visitor needs them.

25. Navigation and Prefetching

Navigation should feel fast without secretly downloading large amounts of media.

- Allow lightweight internal navigation prefetching where beneficial.
- Do not pre-download large full-resolution gallery images simply because a visitor may open them.

- Prefer prefetching lightweight page/data resources over large media.
- Do not preload every gallery image.

26. Admin Application Isolation

The admin system should not make the public website carry the weight of the entire content-management application.

- Admin-only libraries should remain admin-only.
- Rich text editors should not be bundled into public pages.
- Analytics dashboards should not be bundled into public pages.
- Media-management functionality should load only in the admin area.
- Public pages should contain only the code required for the public experience.

27. Performance Targets

These should be treated as engineering targets rather than absolute guarantees. Actual results depend on device, network, geography, traffic and page complexity.

Metric	Target
Largest Contentful Paint	Aim for under 2.5 seconds.
Interaction to Next Paint	Aim for under 200 ms.
Cumulative Layout Shift	Aim for under 0.1.
Duplicate requests	Zero intentional duplicates.
Gallery	Usable immediately without an unnecessary blocking delay.
Images	Delivered at appropriate display dimensions.

28. SEO-Safe Optimization

Performance optimization must not damage Nepalora's search visibility.

- Keep meaningful article text in the HTML where possible.
- Keep proper heading structure.
- Keep internal links.
- Keep useful image alt text.
- Keep metadata.
- Keep canonical URLs.
- Keep sitemap behavior.

- Do not hide important content unnecessarily behind client-only rendering.

29. Accessibility-Safe Optimization

- Meaningful images need useful alt text.
- Decorative images should be treated as decorative.
- Gallery controls should remain keyboard accessible.
- Loading states should not trap keyboard navigation.
- Lazy-loaded content must remain accessible.
- Do not rely solely on hover interactions.

30. Production Optimization Checklist

Cloudinary

- ☐ Automatic format enabled where appropriate.
- ☐ Automatic quality enabled where appropriate.
- ☐ Thumbnail derivatives used for thumbnails.
- ☐ Original images are not used for small cards.
- ☐ Large images load only when necessary.
- ☐ Unused assets removed.
- ☐ Duplicate uploads minimized.
- ☐ Folders are organized.
- ☐ Permanent resized duplicates are minimized.

Next.js

- ☐ Public pages are cache-friendly.
- ☐ Unnecessary dynamic rendering has been removed.
- ☐ Images use responsive sizing.
- ☐ Only critical images load eagerly.
- ☐ Secondary images lazy-load.
- ☐ Heavy client components are isolated.
- ☐ No duplicate fetching.
- ☐ Admin code is separated from public bundles.

Database

- ☐ Gallery queries select only required fields.

- ☐ Relevant filters are indexed.
- ☐ No N+1 query patterns.
- ☐ Stable public content can be cached.
- ☐ Admin/private queries are not publicly cached.

Frontend

- ☐ No unnecessary full-page blocking spinner.
- ☐ Image dimensions are reserved.
- ☐ No duplicate image downloads.
- ☐ Third-party scripts have been audited.
- ☐ Non-critical scripts do not block primary content.

31. Recommended Implementation Order

Do not change everything simultaneously. Implement and measure each category.

#	Task	Reason
1	Identify the exact 5–7 second request.	Prevents optimizing the wrong component.
2	Fix Cloudinary image sizing.	Reduces image transfer and bandwidth.
3	Enable automatic format and quality delivery.	Reduces image payloads.
4	Optimize gallery data fetching.	Reduces server/database latency.
5	Cache public content.	Prevents repeated database/server work.
6	Remove duplicate requests.	Reduces latency and bandwidth.
7	Implement lazy loading appropriately.	Improves initial loading.
8	Reserve image dimensions.	Improves visual stability.
9	Audit JavaScript.	Reduces browser execution/hydration cost.
10	Audit third-party scripts.	Prevents external services from slowing pages.
11	Measure real-world performance.	Prevents regressions.

32. Monitoring After Launch

Optimization is not a one-time task. As Nepalora's content library and traffic increase, performance must be monitored.

Monitor:

- Real-user Core Web Vitals.
- Cloudinary bandwidth usage.
- Cloudinary storage usage.
- Image transformation usage.
- Database query duration.
- Database request volume.
- Vercel/server execution behavior.
- Largest JavaScript bundles.
- Slow API requests.
- Duplicate network requests.

33. Performance Regression Testing

Every major update should be tested before deployment.

1. Build the production version.
2. Deploy to the production environment or a production-like preview.
3. Open the homepage.
4. Open an article.
5. Open the gallery.
6. Open several images.
7. Test mobile viewport.
8. Test a slower network.
9. Inspect Network requests.
10. Check Core Web Vitals.

34. Definition of Done

The optimization project can be considered complete when all of the following are true:

- ☐ The gallery does not contain an unnecessary multi-second blocking delay.
- ☐ The gallery shell becomes usable quickly.
- ☐ Thumbnail images are appropriately sized.
- ☐ Large images are not downloaded unnecessarily.
- ☐ Public content is cacheable or appropriately revalidated.
- ☐ Database queries are efficient.

- ☐ There are no unnecessary N+1 requests.
- ☐ There are no intentional duplicate image/data requests.
- ☐ Admin dependencies are isolated from public pages.
- ☐ Third-party scripts have been audited.
- ☐ Cloudinary contains minimal unnecessary duplicate assets.
- ☐ Mobile image payloads are appropriately reduced.
- ☐ Core Web Vitals do not have major regressions.
- ☐ The existing visual design remains unchanged.
- ☐ The implementation remains understandable for future development.

35. Final Engineering Principle

Do not optimize by simply making everything smaller.

Optimize by making every request appropriate.

A small gallery card should receive a small image. An article should receive an image appropriate for its content width. A full-screen viewer can receive a larger image. Stable public content should be cached. Admin code should remain out of public bundles. Below-the-fold media should not block the initial page.

The result should be a website that feels fast without changing what Nepalora looks like.

36. Practical Long-Term Strategy

Nepalora should be built so that increasing content does not automatically mean increasing complexity at the same rate.

The architecture should allow the site to grow from:

```
10 articles
20 articles
50 articles
100 articles
500 articles
1,000+ articles

and

50 images
500 images
5,000 images
10,000+ images
```

without requiring a fundamental redesign of the frontend.

The combination of optimized Cloudinary delivery, caching, efficient database queries, responsive images, lazy loading and lightweight JavaScript provides the foundation for that growth.

Nepalora Performance Documentation

This document is an implementation guide. Exact Cloudinary, Next.js and Vercel defaults and plan limits may change over time, so production configuration should always be verified against the current official documentation and the actual deployed application's network behavior.